

E6 / E7 results: structured masking, noise, spline params, PC loss

EmbodiedMAE-4M, yongyun branch · status as of 2026-09-22 · mirrors the “E6 / E7 on the yongyun branch” section of CLAUDE.md

Bottom line

- **Occlusion recipes (E6): no winner.** The four blob-size $\times$ noise recipes differ by about as much as the same recipe run twice with a different seed. Recipe B (`length_scale` 0.20, $1\times$ noise) stays the default: best observed, not shown better.
- **The spline params already help.** With every param token visible, swapping a sample’s own params for the val-set mean makes PC Chamfer 7% worse, far above the 0.6% seed noise.
- **The params fix (v2 + camera pose/scale) has not worked yet.** It did not raise reliance on the params, and PC Chamfer got 6% worse. Suspected cause: v2’s param loss is $\sim 6\times$ larger and takes gradient from the PC term. The next run rescales `spline_loss_weight` to ~ 0.6 .
- **The model’s weak point is leaf tips, not the frame edge.** The edge-biased mask does mask the border more ($0.64 \rightarrow 0.89$), but the border is background. On the plant, PC error on the outermost points is $\sim 10\times$ the centre’s.
- **PC loss study (E7): 1 of 11 arms finished.** QAL threshold 0.005 matches the 0.01 baseline within noise. The Chamfer and Sinkhorn arms have not run yet.

Setup shared by every run

- 4M base · 3000 train plants, one randomly drawn view per plant per epoch · **30000 optimizer steps** (94 steps/epoch $\rightarrow$ **320 epochs**) · global batch 32 (8 per GPU $\times$ 4 L40S) · `Sorghum_15K` split. Best checkpoints land at epochs 300–320: still improving when the cosine schedule ends.
- Clean val: a fixed 225-view subset. Occluded val: one reference corruption shared by all runs (`occlusion.eval_occlusion` at $1\times$ noise), so occluded numbers are comparable.
- **Held fixed, including the edge weighting.** Structured token masking on half the batches (`prob` 0.5) with `center_bias` 1.5: masked blobs gather at the frame border and the centre plant stays visible; train-only, so validation uses uniform masks. Procedural occluding leaves enter from outside the frame (`reach` 0.6–1.2, 9–16 leaves, every sample), interleaved in depth, plus sensor noise. **No run varies `center_bias`, `prob` or `reach`**, so these results do not say whether the edge weighting helps.
- 30000 steps is $\sim 15\%$ of the main programme’s 197400-step budget (`e3_3k`, same 94 steps/epoch, runs 2100 epochs): rankings transfer, absolute numbers do not. The 3000 plants are not the same draw as main’s `e3_3k`.

Noise floor: read every table against this

Training had no seed before `training.seed` was added, so `recipe_b` (unseeded) and `params_v1_seed1` (seed 1) are the same configuration run twice. Their gap is the only run-to-run error bar available.

	val loss	val (occ) loss	PC Chamfer	F1@0.01	F1@0.03	EMD	Depth MSE
seed-to-seed gap	0.027	0.014	0.6%	1.7%	0.3%	1.4%	11%

Table 1: One pair of runs, so this is itself a rough estimate. Differences smaller than this are treated as noise.

Which F1 threshold

F1@0.03 is the headline column: its ceiling is ~ 1 , so the value reads directly as a fraction of a perfect reconstruction. F1@0.01 is kept beside it. Its ceiling is well below 1, because two independent 8 196-point draws of the *same* ground-truth cloud only reach:

	F1@0.005	F1@0.01	F1@0.02	F1@0.03
perfect reconstruction (sampling limit)	0.465	0.871	0.995	0.999
our models	–	0.150–0.159	0.403–0.416	0.575–0.586

So F1@0.01 = 0.154 is $\sim 18\%$ of what is reachable. It is also slightly the more discriminating column here (recipe spread / seed gap $3.1\times$ vs $2.7\times$ at 0.03), and it is the one that exposes the leaf-tip failure below. Moving to a threshold where F1 ≈ 0.9 would not help: every arm saturates there and the gaps compress.

E6: structured-mask blob size $\times$ sensor noise

run		blob size	train noise	val loss	val (occ)	PC Chamfer	F1@0.03	F1@0.01	RGB MSE	Depth MSE
A	recipe_a_ls035_noise1x	0.35	1 $\times$	0.5829	0.5964	0.004703	0.5832	0.1582	0.6738	0.03350
B	recipe_b_ls020_noise1x	0.20	1 $\times$	0.5623	0.5853	0.004644	0.5856	0.1567	0.6649	0.03355
C	recipe_c_ls020_noise0p5x	0.20	0.5 $\times$	0.5908	0.5993	0.004716	0.5816	0.1536	0.6715	0.03219
D	recipe_d_ls020_noise2x	0.20	2 $\times$	0.5891	0.6010	0.004680	0.5817	0.1503	0.6610	0.03221
B'	params_v1_seed1 (= B, seed 1)	0.20	1 $\times$	0.5891	0.5996	0.004670	0.5841	0.1541	0.6773	0.02997

Table 2: Blob size = `length_scale`. 1 $\times$ noise = rgb 0.03 / depth 0.015 / pc 0.008 (std). B' is B re-run with only the random seed changed, so B vs B' is the run-to-run noise. Nothing is bold: no gap in this table exceeds that seed-to-seed gap.

B has the best clean and occluded val loss, but **B run a second time (B') lands behind A and level with C and D**. Every recipe gap (A–B 0.021, C–B 0.029, D–B 0.027 in val loss) is the size of the seed gap, so the sweep does not rank these recipes, and the order of A and B flips with the F1 threshold (A > B at 0.01, B > A at 0.02 and 0.03). Separating them needs at least three seeds per arm.

Where the model fails: leaf tips, not the frame edge

Trained recipe-B model, 12 val plants, 4 mask draws. Radius 0 = image centre, 1 = corner (for the PC, the xy radius of the unit-normalised cloud). Errors are on *masked* patches.

radius	0–0.2	0.2–0.4	0.4–0.6	0.6–0.8	0.8–1
plant share of patches	71 %	40 %	16 %	3 %	0.2 %
P(masked), uniform mask	0.64	0.61	0.60	0.60	0.62
P(masked), structured (<code>center_bias</code> 1.5)	0.64	0.70	0.81	0.86	0.89
RGB MSE, all masked patches	0.18	0.17	0.09	0.02	0.002
RGB MSE, plant patches only	0.20	0.25	0.29	0.27	0.60
PC: target point $\rightarrow$ nearest prediction	0.025	0.042	0.072	0.119	0.233

The edge bias does what it says: masking rises from 0.64 to 0.89 toward the corner. But the corner is background, which is why the error there is near zero. On the plant itself, error *grows* outward, and PC error on the outermost points is $\sim 10\times$ the centre's: the model misses leaf tips. That is the low F1@0.01 (precision 0.40, recall 0.11 at the end of training: predicted points sit on the dense core and do not reach the tips). Much of the edge-biased mask budget is therefore spent on background; masking the plant's *outer parts* would target the actual failure.

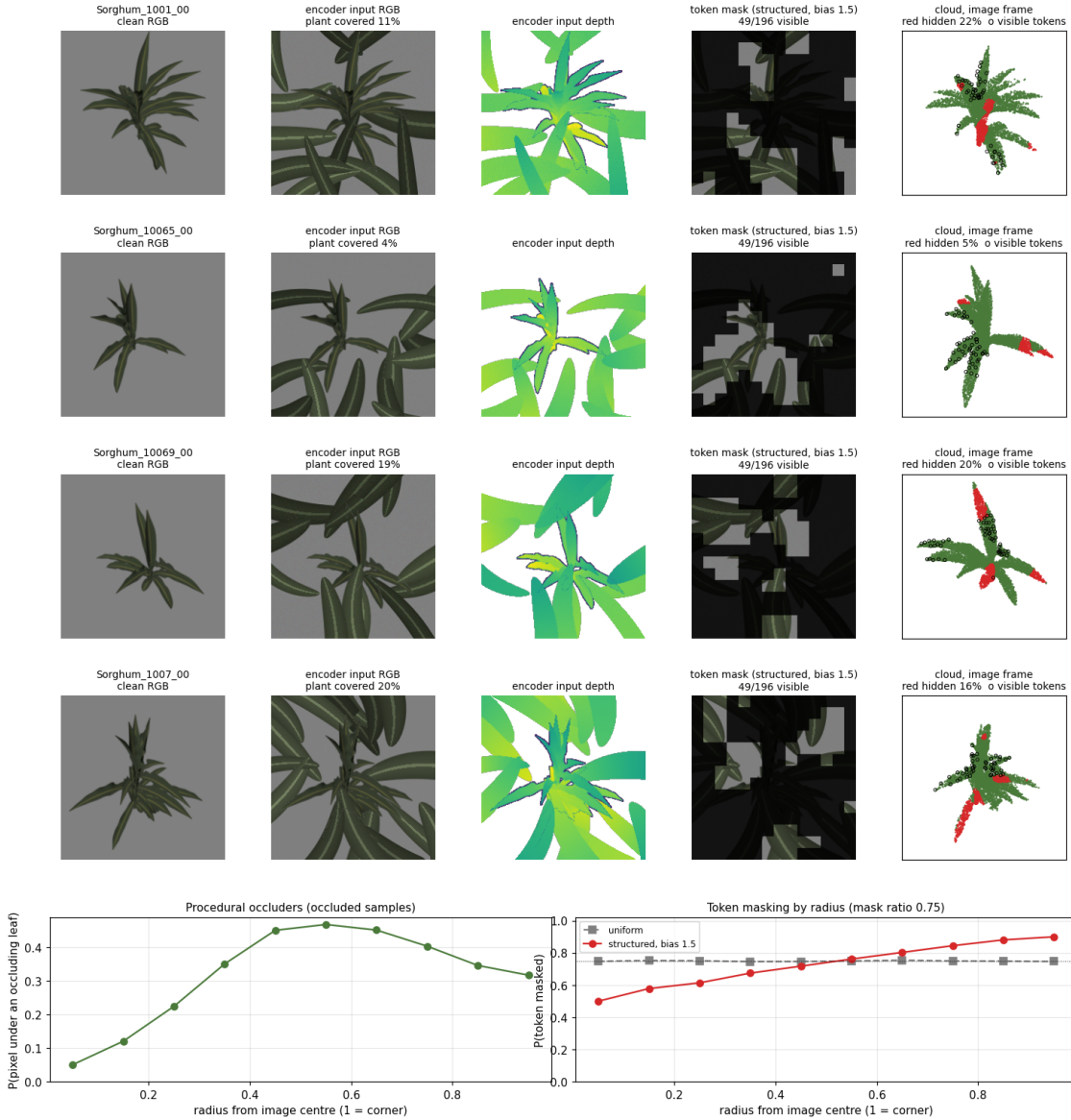


Figure 1: Recipe-B masks on four val plants: clean RGB, occluded encoder input, depth, structured token mask (49 of 196 tokens visible) and the PC with hidden points in red. Bottom: occluder coverage and token-mask probability by radius.

Spline params: do they help, and does fixing their encoding help more?

Measured on the data first: 5 of the 9 plant dims (`panicle_*`) are constant; the per-sample PC normalisation removes absolute size (correlation of `stem_length` with `cloud_height` is 0.68 raw, 0.08 after normalisation); and the params live in the plant's world frame while the PC target is in the camera frame, with the `camera_pose.json` that links them never read. **v2** z-scores the params, encodes angles as $(\sin, \cos)$, drops the constant dims and adds the leaf count; **geometry conditioning** feeds the camera rotation and normalisation radius as an always-visible token. Same seed; only the params path differs.

run	PC Chamfer	F1@0.01	F1@0.02	F1@0.03	EMD	RGB MSE	Depth MSE
v1 <code>params_v1_seed1</code>	0.004670	0.1541	0.4097	0.5841	0.2966	0.6773	0.02997
v2 + pose/scale <code>params_v2cond_seed1</code>	0.004967	0.1587	0.4057	0.5748	0.2948	0.6758	0.03256

Table 3: Bold = better than the other run by more than the seed-to-seed gap (PC Chamfer 0.6 %, F1@0.01 1.7 %, F1@0.03 0.3 %); every other gap is within noise.

Oracle test (`eval_param_oracle.py`): all param tokens visible; the sample’s own params vs the val-set mean, with identical RGB / depth / PC masks in both passes. Positive = worse without the sample’s own params, i.e. the params are being used.

checkpoint	PC Chamfer	F1@0.01	F1@0.02	RGB MSE	Depth MSE	occluded PC Chamfer
v1	+7.0 %	+1.4 %	+2.1 %	−1.7 %	+7.1 %	+7.9 %
v2 + pose/scale	+4.6 %	+5.3 %	+2.5 %	+0.4 %	−0.2 %	+4.5 %

The params already help in v1. v2 + pose/scale did not increase that reliance, and its absolute PC Chamfer is 6 % worse. Most likely cause, not yet isolated: v2’s Smooth-L1 targets are z-scores, so the param loss is $\sim 6\times$ v1’s at the same `spline_loss_weight` of 5. Next: v2 with `spline_loss_weight` ≈ 0.6 ; if still flat, asymmetric masking (steps with the PC fully masked and the params fully visible).

E7: point-cloud loss study (in progress)

Weights were set from loss magnitudes measured on the trained recipe-B model: QAL 0.060, Chamfer 0.0053 ($11\times$ smaller), Sinkhorn ≈ 0.055 . Chamfer therefore needs `pc_loss_weight` ≈ 10 to pull as hard as QAL at 1. “Sinkhorn only” is a new option (`loss_name: sinkhorn`, on 2048 sampled points); before, Sinkhorn existed only as an auxiliary term. Everything else matches `params_v1_seed1`, which serves as the QAL baseline arm.

loss	settings	runs	status
QAL	threshold 0.005 / 0.01 / 0.02 at $\alpha = 100$; $\alpha = 30$ / 300 at threshold 0.01	<code>loss_q_t005</code> , <code>params_v1_seed1</code> , <code>loss_q_t020</code> , <code>loss_q_a030</code> , <code>loss_q_a300</code>	2 done, 1 running, 2 queued
Chamfer	<code>pc_loss_weight</code> 3 / 10 / 30	<code>loss_c_w03</code> , <code>loss_c_w10</code> , <code>loss_c_w30</code>	queued
Sinkhorn only	blur 0.01 / 0.02 / 0.05 at weight 1; blur 0.02 at weight 3	<code>loss_s_b010_w1</code> , <code>loss_s_b020_w1</code> , <code>loss_s_b050_w1</code> , <code>loss_s_b020_w3</code>	queued

Finished so far, compared only on metrics computed identically for every arm (val loss is a different quantity per loss):

run	setting	PC Chamfer	F1@0.01	F1@0.02	F1@0.03	EMD	RGB MSE	Depth MSE
<code>params_v1_seed1</code>	QAL $t = 0.01$, $\alpha = 100$	0.004670	0.1541	0.4097	0.5841	0.2966	0.6773	0.02997
<code>loss_q_t005</code>	QAL $t = 0.005$, $\alpha = 100$	0.004700	0.1540	0.4060	0.5822	0.2970	0.6910	0.02898

Table 4: Halving the QAL threshold moves nothing past the noise floor.

Progress: the `mech-ai` account is at its 17-GPU cap, so the arms run one at a time (~ 5 h each, 4 GPUs per arm). At that pace the last arm finishes around Sep 24. Full table: `python compare_recipes.py -loss-study`.

Before merging with main

- New config keys, all off by default (with none set, a model is bit-identical to before): `training.num_train_plants`, `views_per_epoch`, `max_steps`, `seed`; `occlusion.eval_occlusion`; `model.param_encoding`, `geometry_cond`; `loss_name: sinkhorn`.
- Main implements plant subsetting and view sampling separately (`data.max_plants`, `data.view_sampling`). Keep one when merging; they will conflict in `train_sorghum_4m.py` and the datasets.